

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science, Semester V
Subject	CS3807 – Deep Learning Laboratory
Name	R S kirithic
Section	AIDS B
Register Number	24011101082

Experiment 1 Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The goal of this experiment is to build a Single Layer Perceptron from scratch (no major libraries imported) for binary classification. We have to use the perceptron learning rule, the activation function, plotting the learning process, and checking the classifier on a real dataset.

2. Tasks Performed

1. Downloaded and explored the Banknote Authentication dataset.
2. Performed exploratory data analysis (EDA) on the dataset.
3. Preprocessed the dataset, normalized the features and split it into train and test sets.
4. Implemented a Single Layer Perceptron from scratch.
5. Trained the model using the perceptron learning rule and logged the training progress.
6. Evaluated the classifier using accuracy, precision, recall, F1-score and a confusion matrix.
7. Compared the Step and Sigmoid activation functions.
8. Studied the effect of different learning rates on convergence.
9. Compared the from-scratch implementation with scikit-learn's Perceptron.
10. Showed why a single layer perceptron cannot solve the XOR problem.
11. Studied the effect of feature normalization on convergence.

3. Background Theory

An artificial neuron takes one or more input features, multiplies them by weights, adds a bias, and passes the result through an activation function to give the output.

Weighted sum:

$$z = w^T x + b$$

Prediction:

$$\hat{y} = f(z)$$

Weight update rule:

$$w_{new} = w_{old} + \eta(y - \hat{y})x$$

Bias update:

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

η i: learning rate. The Single Layer Perceptron is a supervised learning algorithm and it can only solve problems where the classes are linearly separable.

4. Activation Functions

Step Activation Function

The perceptron used in this experiment uses only the Step activation function:

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

This gives a hard 0 or 1 output and only works for linearly separable problems.

Activation	Output Range	Typical Use
Step	{0, 1}	Classical Perceptron
Sigmoid	(0, 1)	Binary Classification Output Layer
Tanh	(-1, 1)	Hidden Layers (Traditional Networks)
ReLU	[0, ∞)	Hidden Layers in CNNs and MLPs
Leaky ReLU	(-∞, ∞)	Avoiding Dying ReLU
Softmax	(0, 1)	Multi-class Classification Output

5. Dataset Description

Dataset: Banknote Authentication Dataset (UCI Machine Learning Repository).

Instances	1372
Features	4 numerical features
Classes	2
Missing Values	None
Task	Binary Classification

Features: variance, skewness, curtosis, entropy of the wavelet transformed image.

Target class: 0 for authentic banknote, 1 for forged banknote.

6. Experimental Procedure and Results

Task 1: Dataset Exploration

I loaded the dataset with pandas and printed the first five rows, the dataset shape, the missing value count and the descriptive statistics.

```
First five samples:
   variance  skewness  curtosis  entropy  class
0   3.62160    8.6661  -2.8073  -0.44699      0
1   4.54590    8.1674  -2.4586  -1.46210      0
2   3.86600   -2.6383   1.9242   0.10645      0
3   3.45660    9.5228  -4.0112  -3.59440      0
4   0.32924   -4.4552   4.5718  -0.98880      0
```

Dataset dimensions: 1372 rows x 5 columns

Missing values per column:

```
variance    0
skewness    0
```

```

curtosis    0
entropy     0
class       0

```

Descriptive statistics:

Stat	variance	skewness	curtosis	entropy
mean	0.4337	1.9224	1.3976	-1.1917
std	2.8428	5.8690	4.3100	2.1010
min	-7.0421	-13.7731	-5.2861	-8.5482
25%	-1.7730	-1.7082	-1.5750	-2.4135
50%	0.4962	2.3197	0.6166	-0.5867
75%	2.8215	6.8146	3.1793	0.3948
max	6.8248	12.9516	17.9274	2.4495

Class distribution: 762 good notes (class 0) and 610 forged notes (class 1). The dataset is balanced, so we don't need an algorithm for class weighting .

Task 2: Exploratory Data Analysis

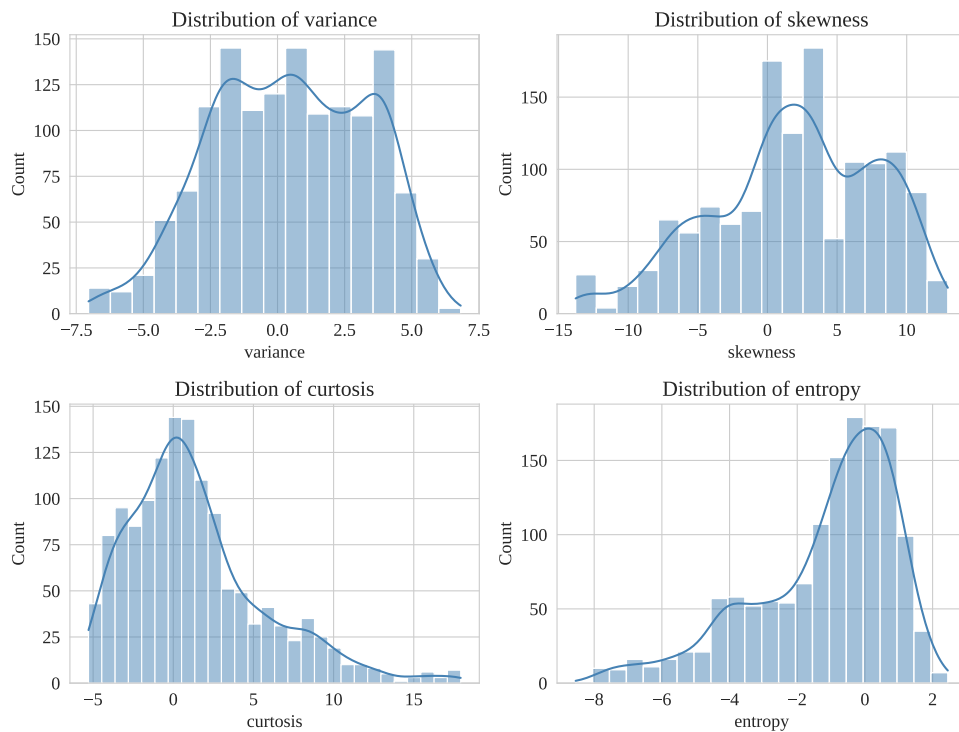


Figure 1: Feature Histograms.

Observation: the two classes form largely separate clusters along the variance and skewness axes, so the problem looks close to linearly separable.

Observation: variance shows the clearest separation between the two classes out of all four features, which matches what is seen in the scatter plots.

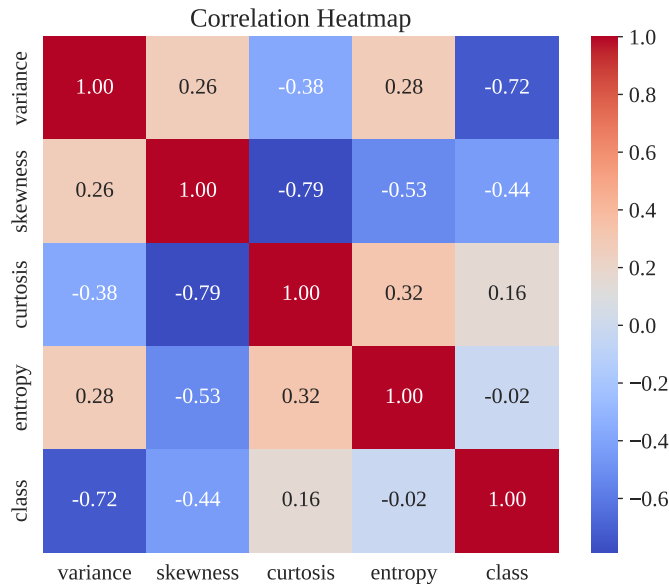


Figure 2: Correlation Heatmap.

Task 3: Data Preprocessing

All four features were normalized using `StandardScaler` (zero mean, unit variance), and the data was split into 80% training and 20% testing, using stratified sampling to keep the class balance in both sets.

```
Training set size: 1097 samples
Testing set size : 275 samples
Feature mean after scaling (train): [ 0.01 -0.008 0.003 -0.01 ]
Feature std after scaling (train): [0.999 1.006 1.011 1.005]
```

Task 4: Perceptron Implementation (From Scratch)

The perceptron was implemented from scratch with the following parts:

- Weight initialization: zeros by default.
- Bias initialization: zero by default.
- Step activation function.
- Forward propagation: $z = w^T x + b$, then apply the step function.
- Learning rule: $w_{new} = w_{old} + \eta(y - \hat{y})x$, $b_{new} = b_{old} + \eta(y - \hat{y})$.

Task 5: Model Training

The model was trained with learning rate 0.01 for 50 epochs. Each epoch printed the number of misclassified samples, the updated weights and the updated bias.

Epoch-wise training log (selected epochs, full log has 50 rows):



Figure 3: Pairwise Scatter Plots of All Features, split by class.

Epoch	Errors	w_1	w_2	w_3	w_4	Bias
1	59	-0.0680	-0.0938	-0.0678	-0.0110	-0.03
5	20	-0.0991	-0.1295	-0.1037	0.0102	-0.05
10	20	-0.1307	-0.1541	-0.1250	-0.0060	-0.06
20	19	-0.1636	-0.1946	-0.1613	-0.0064	-0.07
30	17	-0.1751	-0.2043	-0.1901	-0.0196	-0.08
40	18	-0.1948	-0.2244	-0.1853	-0.0197	-0.09
50	18	-0.1941	-0.2358	-0.2109	-0.0195	-0.10

(w_1, w_2, w_3, w_4 are the weights for variance, skewness, curtosis and entropy in that order.)

Observation: The misclassifications initially drop a lot, from 59 down to around 15 to 20, and then kind of oscillate for the rest of training instead of reaching zero. The minimum reached was 12 errors, at epochs 23, 35 and 39, but the model never converges fully in 50 epochs, meaning the two classes are close to but not perfectly linearly separable.

Observation: the bias moves from 0 down to about -0.10 in small steps over the 50 epochs and does not swing back and forth much, so it stabilizes along with the weights.

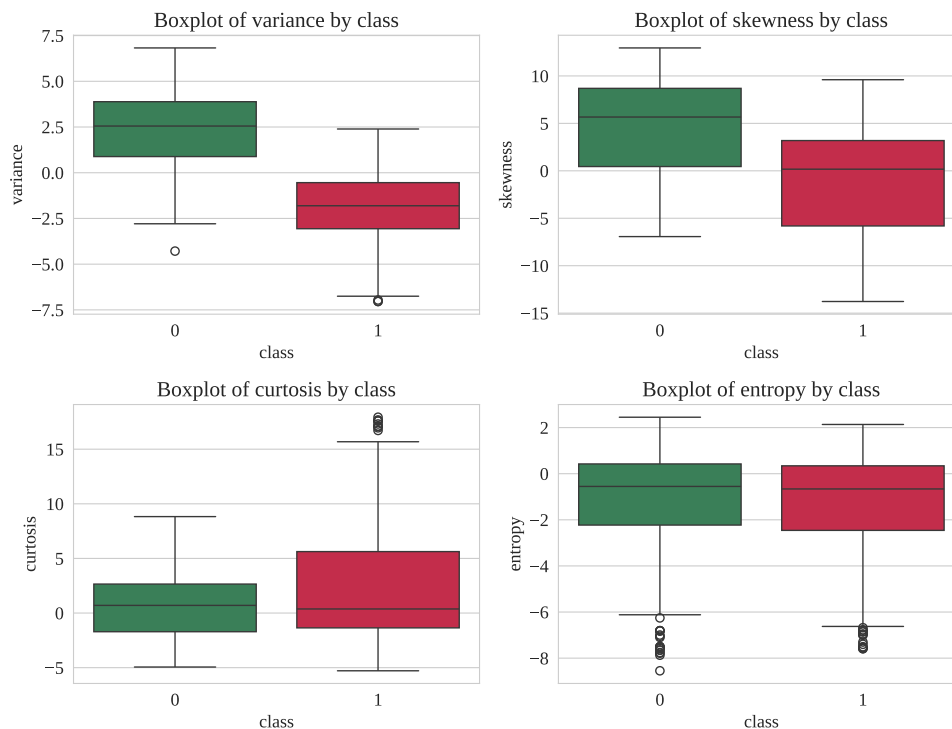


Figure 4: Feature Boxplots by Class.

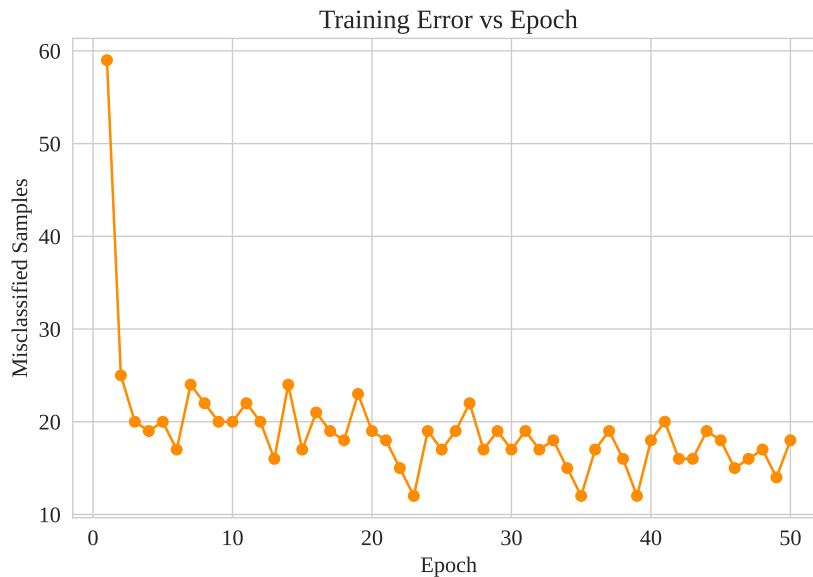


Figure 5: Training Error vs. Epoch.

Task 6: Model Evaluation

On the 275-sample test set:

Accuracy : 0.9855
Precision : 0.9683
Recall : 1.0000

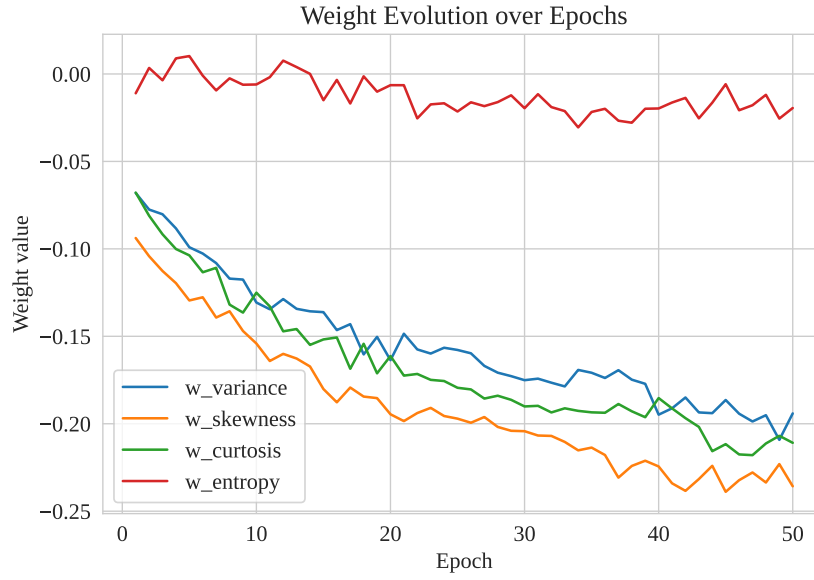


Figure 6: Weight Evolution over Epochs.

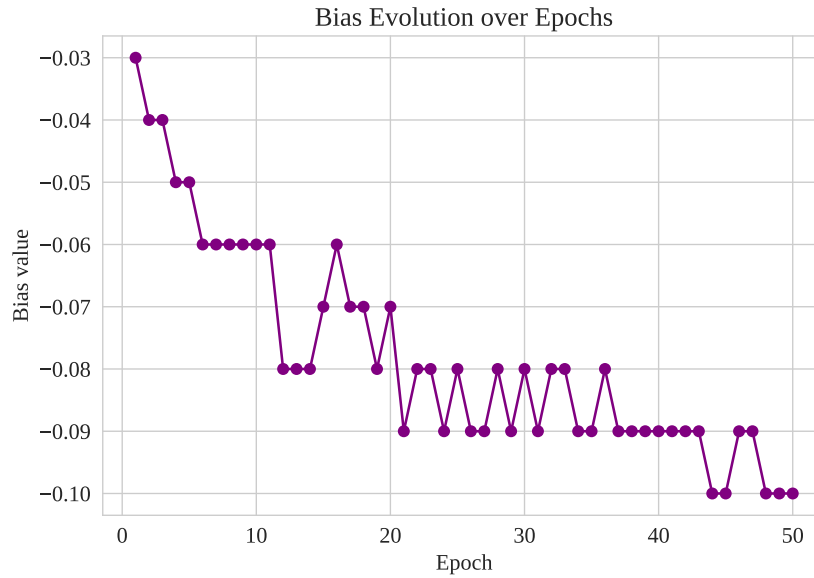


Figure 7: Bias Evolution over Epochs.

F1-score : 0.9839
 Confusion Matrix:
 [[149 4]
 [0 122]]

Out of 275 test samples, 149 authentic notes were correctly classified, 4 authentic notes were wrongly classified as forged, no forged notes were missed, and 122 forged notes were correctly classified.

Observation: the model gets a perfect recall of 1.0, meaning it never misses a forged note in the test set. All 4 mistakes are false positives, authentic notes marked as forged, which is the safer kind of error to make in this problem. Since it means that we never let a forged one be marked as real one which is what the priority should be.

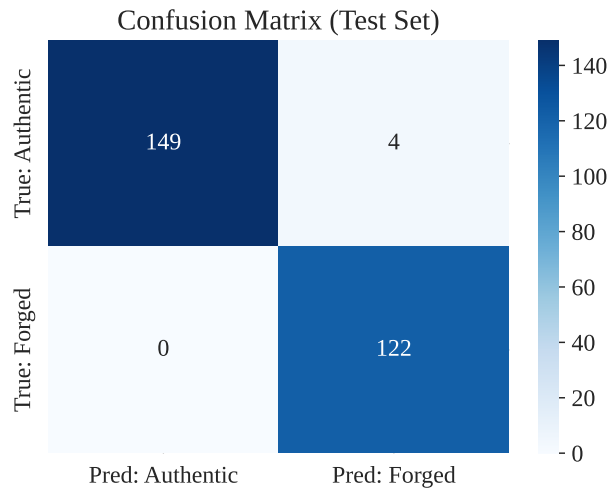


Figure 8: Confusion Matrix (Test Set).

Task 7: Additional Task 1, Step vs Sigmoid Activation

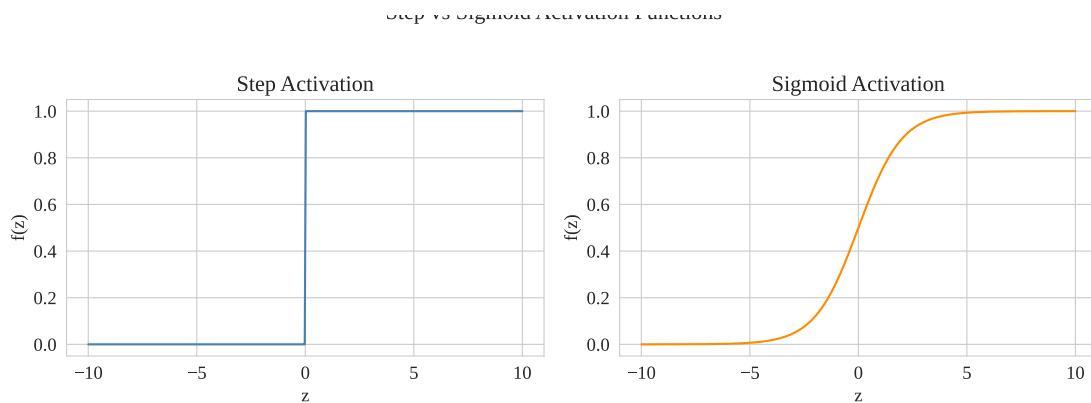


Figure 9: Step vs. Sigmoid Activation Functions.

Discussion: The Step function gives a hard 0 or 1 output with a sudden jump at $z = 0$. Its derivative is zero everywhere except at that one point, so it does not carry any useful gradient information. The Sigmoid function gives a smooth (0,1) output and is differentiable everywhere, with derivative $f'(z) = f(z)(1 - f(z))$.

Task 8: Learning Rate Comparison

With zero weight initialization, the sign of $w \cdot x + b$, and therefore the whole sequence of misclassifications, does not change with the learning rate. To make the comparison meaningful, all three runs below start from the same small random initial weights and bias instead of zero.

Learning Rate	Final Misclassified (epoch 50)	Test Accuracy
0.001	9	0.9891
0.01	13	0.9855
0.1	17	0.9818

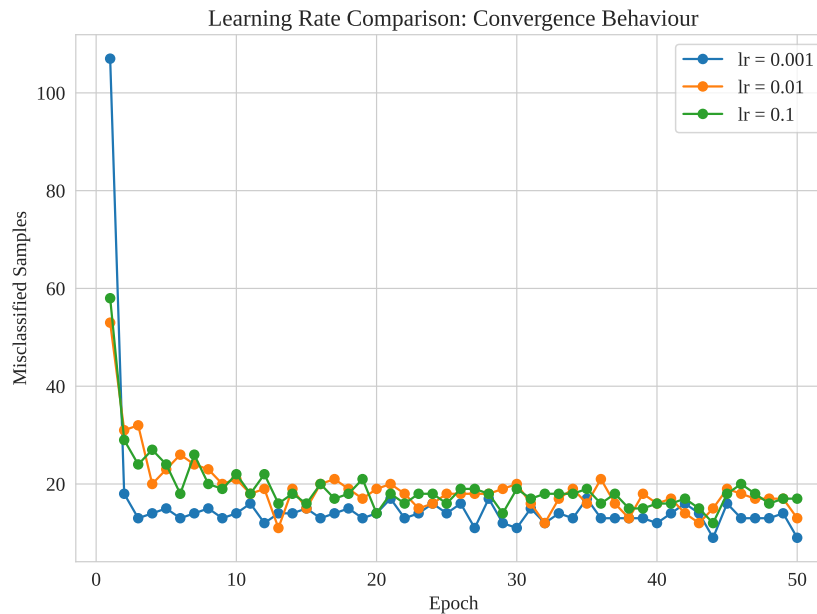


Figure 10: Learning Rate Comparison: Convergence Behaviour.

Observation: the smallest learning rate, 0.001, gives the fewest final misclassifications and the best test accuracy. Larger learning rates make bigger jumps at each update, which pushes the decision boundary past the minima more often near the end of training and gives a slightly noisier, less accurate result here. It also has a tendency to oscillate more .

Task 9 (Optional): Decision Boundary Visualization

A second perceptron was trained using only two features, variance and skewness, so the decision boundary could be drawn in 2D. Test accuracy with just these two features was 0.8364, lower than the 0.9855 achieved using all four features.

Observation: the perceptron learns one straight line that separates most of the authentic and forged notes using only these two features, but some points from each class fall on the wrong side, since two of the four features are being ignored here.

Task 10 (Optional): Comparison with Scikit-learn's Perceptron

Metric	From-Scratch Perceptron	Scikit-learn Perceptron
Accuracy	0.9855	0.9745
Precision	0.9683	0.9457
Recall	1.0000	1.0000
F1-score	0.9839	0.9721

Observation: the from-scratch implementation slightly outperforms scikit-learn's Perceptron on this run, though both reach the same perfect recall. The small gap is most likely because of differences in default settings, such as `max_iter` and the exact stopping rule, rather than any real difference in the learning algorithm itself.

Task 11: Additional Task, Why a Single Layer Perceptron Cannot Solve XOR

A perceptron was trained on the XOR truth table: $(0,0) \rightarrow 0$, $(0,1) \rightarrow 1$, $(1,0) \rightarrow 1$, $(1,1) \rightarrow 0$.

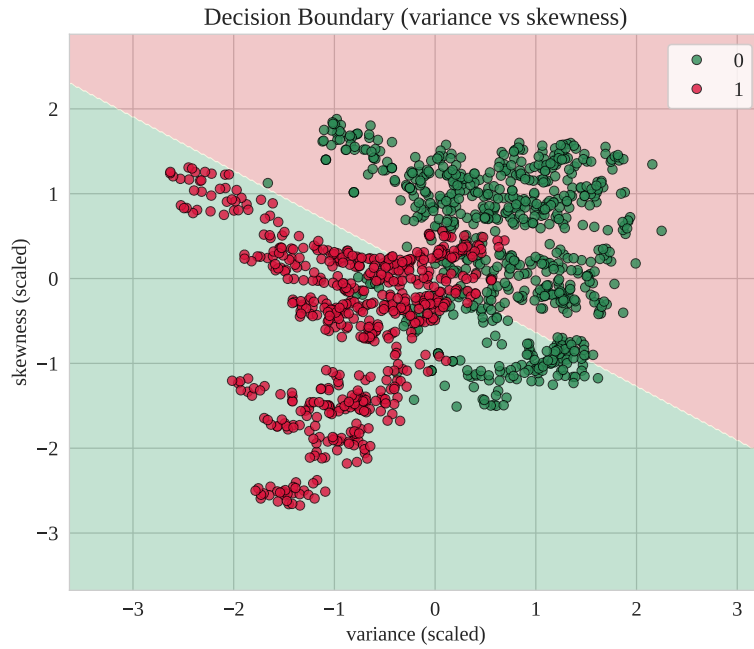


Figure 11: Decision Boundary (variance vs. skewness).

Final misclassifications on XOR after 50 epochs: 4 out of 4 samples

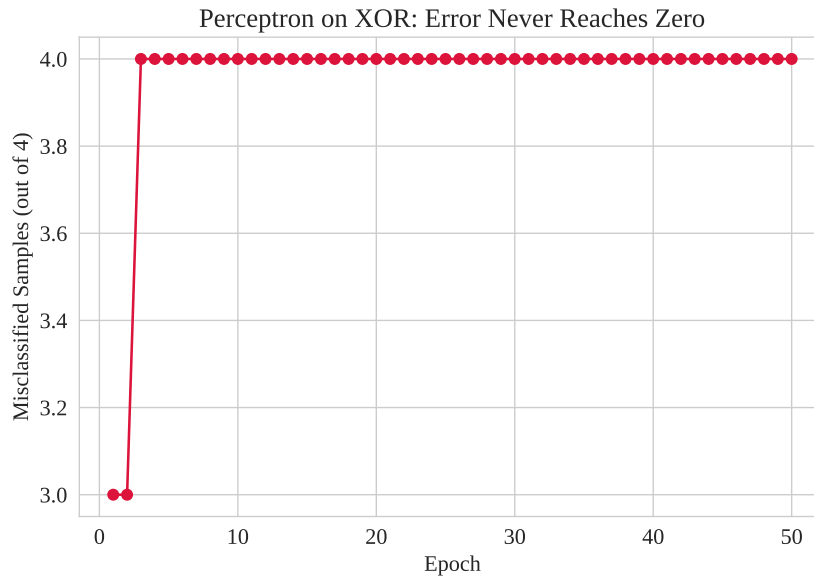


Figure 12: Perceptron on XOR: Error Never Reaches Zero.

Observation: the perceptron never learns XOR, it stays at 4 out of 4 misclassified even after 50 epochs. This is because it has to fit a straight line to separate it but no straight line can (impossible) so we the two positive points $(0, 1)$ and $(1, 0)$ sit on opposite corners from each other and from the two negative points. Separating them needs at least two lines, which means at least one hidden layer, so XOR needs a multi-layer network to solve. SO a non linear boundary for us to separate them .

Task 12: Additional Task, Effect of Feature Normalization on Convergence

Without normalization -> epoch-1 errors: 63, mean errors (epochs 2-10): 23.1,
mean errors (last 20 epochs): 15.2
With normalization -> epoch-1 errors: 59, mean errors (epochs 2-10): 20.8,
mean errors (last 20 epochs): 16.6

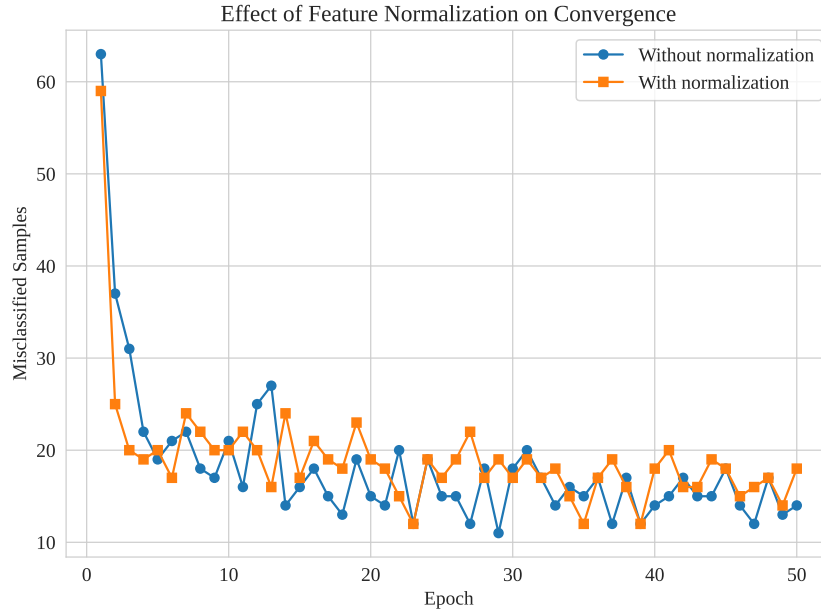


Figure 13: Effect of Feature Normalization on Convergence.

Observation: normalization gives slightly fewer errors early in training (epoch 1 and the epochs 2 to 10 average). However there isn't much difference.

7. Performance Tables

Training Summary

Dataset Size	1372
Train/Test Split	80/20 (stratified), 1097 train, 275 test
Learning Rate	0.01
Epochs	50
Final Weights	[-0.1941, -0.2358, -0.2109, -0.0195]
Final Bias	-0.10
Accuracy	0.9855
Precision	0.9683
Recall	1.0000
F1-score	0.9839

8. Discussion

1. Was the dataset linearly separable?

yes but not fully . The training error drops fast in the first few epochs and then settles between 12 and 20 misclassified samples out of 1097 training samples, and never reaches zero in 50 epochs. This means most of the data is linearly separable but a small number of points from the two classes overlap. This is normal with real world data.

2. Effect of learning rate.

The smallest learning rate tried, 0.001, gave the best test accuracy (0.9891) and the fewest final errors. Larger learning rates converge in a similar number of epochs but end up slightly less accurate here, since bigger steps overshoot the boundary more near the end of training.

3. From-scratch vs scikit-learn.

Both implementations reached perfect recall (1.0) and very close accuracy, 0.9855 against 0.9745. This is a good sign that the scratch implementation matches the standard algorithm correctly.

4. Step vs Sigmoid.

The Step function cannot be used in deep, multi-layer networks because it blocks gradient flow during backpropagation. Differentiable activations like Sigmoid are needed for that.

5. XOR.

A single layer perceptron can only draw one straight decision boundary, so it cannot solve XOR, which needs at least two lines to separate the classes. This is the classic limitation that shows why multi-layer networks are needed, which will be covered in the next experiment.

6. Normalization.

Normalizing the features helped a bit in the early epochs of training but did not change the final accuracy by much in this case, since the four raw features were not on wildly different scales to begin with.

9. Conclusion

- A Single Layer Perceptron was implemented from scratch using the Rosenblatt learning rule and the Step activation function, and applied to the Banknote Authentication dataset.
- With learning rate 0.01 and 50 epochs, the model reached **98.55% test accuracy**, with perfect recall and only 4 false positives out of 275 test samples.
- Learning rate 0.001 gave the best result out of the three rates tried, reaching 98.91% test accuracy.
- The from-scratch model performed close to, and slightly better than, scikit-learn's Perceptron on this dataset and split.
- The perceptron completely failed on the XOR problem, staying at 4 out of 4 misclassified samples throughout training, which shows the basic limitation of a single linear decision boundary.
- Feature normalization gave a small improvement early in training but did not change the final accuracy much for this dataset.
- Overall the experiment showed how a single artificial neuron learns through weight and bias updates .

10. References

1. F. Rosenblatt, "The Perceptron," Psychological Review, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.

Multi-Layer Perceptron Learning Algorithm for the XOR Gate

Objective

A small multi-layer network was built for the XOR gate: 2 inputs, 2 hidden neurons with step activation, and 1 output neuron with step activation. All three neurons were trained using the plain perceptron delta rule, the same rule used for the single layer perceptron earlier in this report, with no backpropagation. The weights and bias were printed after every single update, the decision boundary was plotted after every update, and the pattern that stops the network from converging was studied.

Training Log

```
Epoch 1, sample [0 0], target 0, pred 1 ->
    W_hidden=[[-0.025, 0.09], [0.046, 0.02]], b_hidden=[-0.169, -0.169],
    W_out=[-0.088 0.073], b_out=-0.080
Epoch 1, sample [0 1], target 1, pred 0 ->
    W_hidden=[[-0.025, 0.19], [0.046, 0.12]], b_hidden=[-0.069, -0.069],
    W_out=[-0.088 0.073], b_out=0.020
Epoch 1, sample [1 1], target 0, pred 1 ->
    W_hidden=[[-0.125, 0.09], [-0.054, 0.02]], b_hidden=[-0.169, -0.169],
    W_out=[-0.188 -0.027], b_out=-0.080
epoch 1 misclassified: 3

Epoch 2, sample [0 1], target 1, pred 0 ->
    W_hidden=[[-0.125, 0.19], [-0.054, 0.12]], b_hidden=[-0.069, -0.069],
    W_out=[-0.188 -0.027], b_out=0.020
Epoch 2, sample [1 1], target 0, pred 1 ->
    W_hidden=[[-0.225, 0.09], [-0.154, 0.02]], b_hidden=[-0.169, -0.169],
    W_out=[-0.188 -0.027], b_out=-0.080
epoch 2 misclassified: 2

Epoch 3, sample [0 1], target 1, pred 0 ->
    W_hidden=[[-0.225, 0.19], [-0.154, 0.12]], b_hidden=[-0.069, -0.069],
    W_out=[-0.188 -0.027], b_out=0.020
Epoch 3, sample [1 1], target 0, pred 1 ->
    W_hidden=[[-0.325, 0.09], [-0.254, 0.02]], b_hidden=[-0.169, -0.169],
    W_out=[-0.188 -0.027], b_out=-0.080
epoch 3 misclassified: 2

...

Epoch 30, sample [0 1], target 1, pred 0 ->
    W_hidden=[[-2.925, 0.19], [-2.854, 0.12]], b_hidden=[-0.069, -0.069],
    W_out=[-0.188 -0.027], b_out=0.020
Epoch 30, sample [1 1], target 0, pred 1 ->
    W_hidden=[[-3.025, 0.09], [-2.954, 0.02]], b_hidden=[-0.169, -0.169],
    W_out=[-0.188 -0.027], b_out=-0.080
epoch 30 misclassified: 2

Misclassified samples per epoch:
[3, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]
```

Decision Boundary Plots

Analysis: The Pattern That Prevents Convergence

The error count drops from 3 to 2 misclassified samples after the first epoch and then gets stuck at exactly 2 for all the remaining 29 epochs. The network never reaches 0 errors.

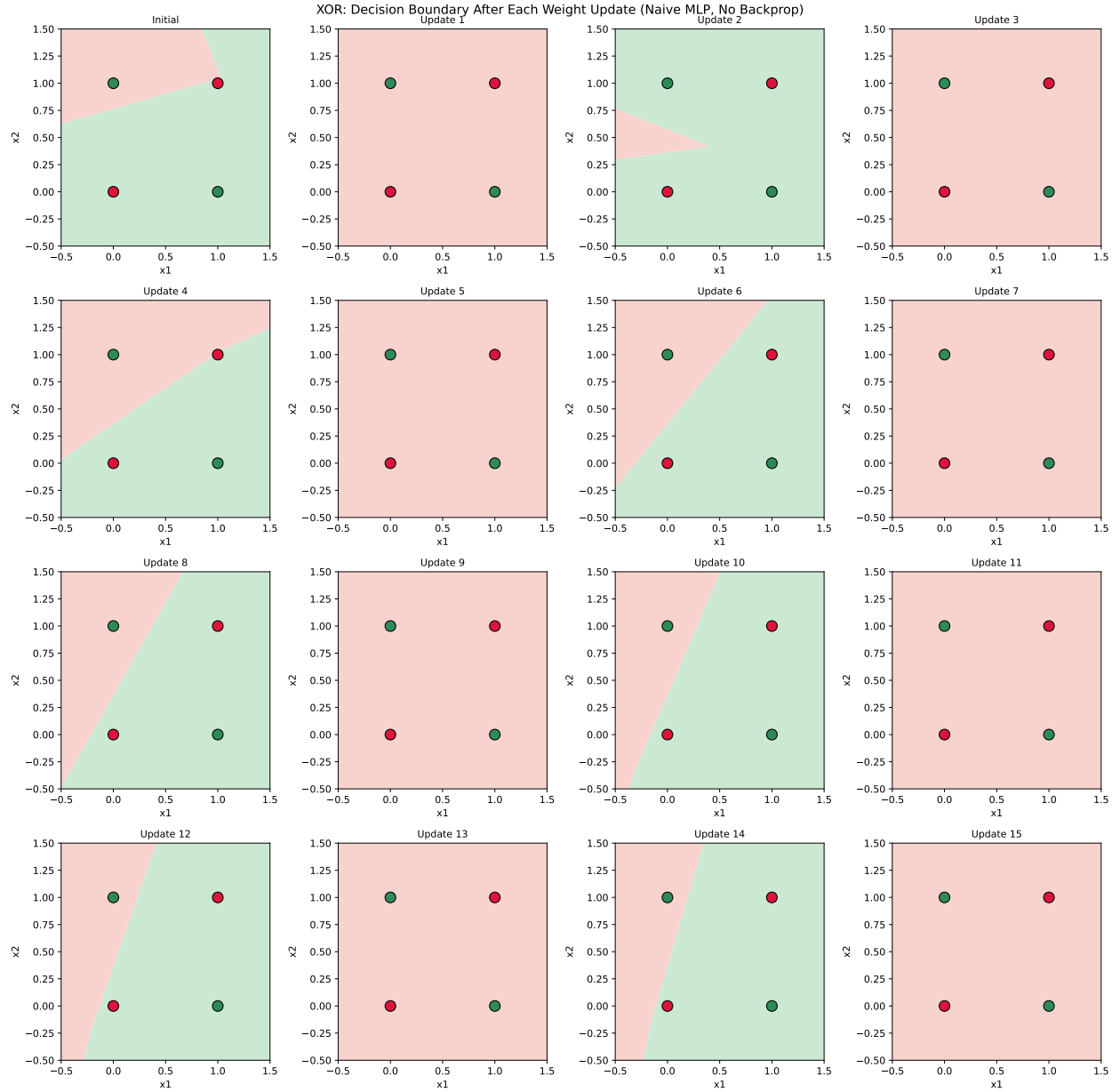


Figure 1: XOR: Decision Boundary After Each Weight Update (Naive Multi-Layer Perceptron, No Backpropagation).

Looking at the weight log, the same two samples, $(0,1)$ and $(1,1)$, are misclassified in every epoch after the first. Each time, the hidden and output weights are updated in the same direction as before, so the hidden layer weights keep growing in size (for example the first hidden weight goes from about -0.03 after epoch 1 to about -3.0 by epoch 30), but the output weights, W_{out} , stop changing completely after epoch 2 and stay fixed at $[-0.188, -0.027]$.

This happens because of how the naive rule updates the hidden layer. The hidden neurons use a step activation, so their output is only ever 0 or 1, no matter how large the hidden weights get. Once the sign of the hidden neuron's weighted sum stops flipping for a given input, that hidden neuron's activation for that input is frozen, even though the raw weight values keep climbing. Since the output weights only ever see 0s and 1s coming out of the hidden layer, once the hidden activations settle into a fixed pattern for the two problem inputs, the output neuron's decision for those two inputs is frozen too. From that point on, every update just makes the weights bigger

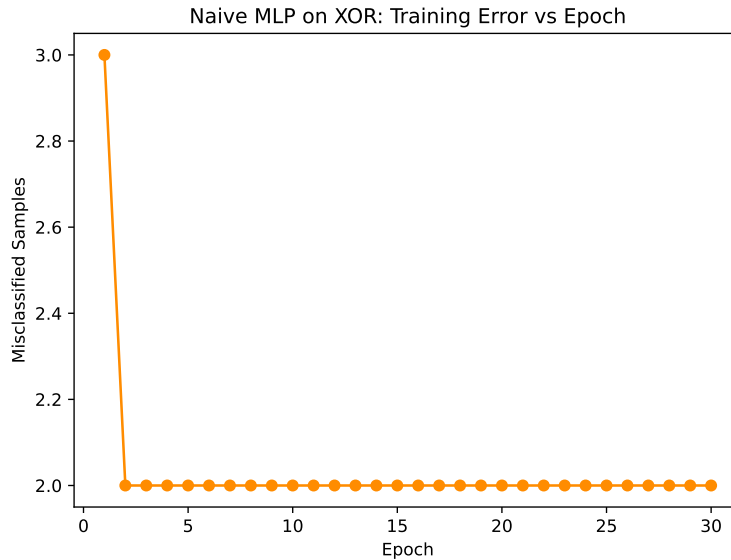


Figure 2: Naive Multi-Layer Perceptron on XOR: Training Error vs Epoch.

in the same direction without ever flipping any sign, so the same 2 samples stay wrong forever.

The deeper reason this happens is that the naive rule updates every hidden neuron using the same output error, with no way to tell each hidden neuron how much it personally contributed to that error. A proper multi-layer network needs backpropagation, which uses the chain rule to work out a separate error term for each hidden neuron based on the output weight connected to it and the derivative of its own activation function. Step activation has a derivative of 0 almost everywhere, so even if backpropagation were applied here, no gradient could flow back through it. This is the same limitation discussed earlier in this report for Step vs Sigmoid: without a differentiable activation and a proper backpropagated error term for each hidden neuron, a multi-layer network has no way to learn a hidden representation that actually helps solve a non-linearly separable problem like XOR, so training gets stuck at a fixed, incorrect solution instead of converging.